# Compact Hash Tables Using Bidirectional Linear Probing

JOHN G. CLEARY

*Abstract* — **An algorithm is developed which reduces the memory requirements of hash tables. This is achieved by storing only a part of each key along with a few extra bits needed to ensure that all keys are stored unambiguously. The fraction of each key stored decreases as the size of the hash table increases. Significant reductions in total memory usage can be achieved especially when the key size is not much larger than the size of a memory index and when only a small amount of data is stored with each key. The algorithm is based on bidirectional linear probing. Search and insertion times are shown by simulation to be similar to those for ordinary bidirectional linear probing.**

*Index Terms* — **Address calculation, bidirectional linear probing, hash storage, information retrieval, memory compaction, open addressing, performance analysis, scatter storage, searching.**

## I. Introduction

THE retrieval of a single item from among many others is a common problem in computer science. I am particularly concerned here with the case where the item is retrieved on the basis of a single label or key attached to each entry and where the keys are not ordered in any particular way. There is a well-known solution to this problem in the form of hash tables. Knuth [8], Knott [7], and Maurer and Lewis [11] provide good introductions to this subject.

An efficient version of hashing called *bidirectional linear probing* (BLP) was developed by Amble and Knuth [1]. As it forms the basis of what follows, it is described in more detail in the following section. Section III shows how it can be modified so as to significantly reduce its memory requirements. This is done by storing only a small part of each key; a few extra bits are needed to ensure that different keys, which look the same after truncation, are correctly distinguished.

The execution time of this compact hashing algorithm is considered in Section IV. It is shown by simulation to be similar to ordinary BLP for both successful searches and insertion. It is significantly better for unsuccessful searches.

A hashing scheme similar to compact hashing in that not all of the key is stored has been proposed by Andreae [2, ch. 1]. However, his technique has a small but finite probability of retrieving an incorrect key. Although compact hashing is not based on this earlier technique, it provided the impetus to seek the current solution.

Manuscript received December 13, 1982; revised September 1, 1983 and March 23, 1984. This work was supported by the Natural Sciences and Engineering Research Council of Canada.

The author is with the Man–Machine Systems Group, Department of Computer Science, The University of Calgary, Calgary, Alta., Canada.

In hashing algorithms using an overflow area and a linked list of synonyms or by variations of this using buckets (see Maurer and Lewis [11]), only the remainder of each key need be stored. This has been known since at least 1965 (Feldman and Low [6] and Knuth [8, sec. 6.4, p. 543, ex. 13]). However, each entry (including the original hash location) requires a pointer to the next overflow record. This pointer will be about the same size as the reduction in the key size. So, there is no net memory saving over open addressing techniques such as BLP.

Amongst the possible applications of compact hashing is the storage of trees and TRIES without the use of pointers but still preserving a log $N$ retrieval time. It is hoped to report on this application in more detail later.

Pascal versions of the algorithms described below are available from the author.

## II. Bidirectional Linear Probing

I will now introduce the open addressing technique which forms the basis of compact hashing. The *hash table* in which the keys will be stored is an array $T[0 \cdots M-1]$. I will be concerned only with the keys themselves as the items associated with each key do not significantly affect the algorithms. In order to compute the location for each key, I will use two functions: $t$ which randomizes the original keys, and $h$ which computes a value in the range $0 \cdots M-1$.

Let $\mathbf{K}$ be the set of all possible keys and $\mathbf{H}$ be the set of all possible transformed keys. Then $t:\mathbf{K} \rightarrow \mathbf{H}$ is an invertible function. This function is introduced to ensure that the keys stored are random and so, as a consequence, the hashing procedure has a satisfactory average performance. In what follows, these transformed keys will be used rather than the original keys. For example, it is the transformed keys that are stored in $T$. ($-1$ is used to indicate an unoccupied location in $T$.)

$h:\mathbf{H} \rightarrow \{0 \cdots M-1\}$ and has the property that for $H_1, H_2 \in \mathbf{H}$ $H_1 \leq H_2$ **iff** $h(H_1) \leq h(H_2)$. As a consequence, the keys will be mapped into the hash table in the same order as the values of their transformed keys. This ordering is essential to the compaction attained later. Suitable functions $t$ and $h$ have been extensively discussed (Carter and Wegman [3]; Knott [7]; Lum [9]; Lum, Yuen, and Dodd [10]). These authors show that there are functions which almost always make the distribution of transformed keys random. I will not consider any particular functions for $t$, although some examples of $h$ will be introduced later.

0018-9340/84/0900-0828$01.00 

To retrieve a key $K$ from the hash table the transformed key and the hash location are first computed. If the (transformed) key stored at the hash location is greater than $t(K)$, then the table is searched upward until one of three things happens. Either an empty location will be found, $T[j] = -1$, or the sought key will be found, $T[j] = t(K)$, or a key greater than the sought key will be found, $T[j] > t(K)$. If the first key examined is less than $t(K)$ then an analogous search is done down the hash table. The search is successful if the sought key is found, that is if the last location examined is equal to $t(K)$, and is unsuccessful otherwise. (See Amble and Knuth [1] for the details of this algorithm.)

For a given set of keys there are many ways that it can be arranged in $T$ so that the search algorithm above will still work correctly. There is thus freedom, when designing an algorithm to insert new keys, to choose different strategies for positioning the keys. There are two conditions that must be satisfied when a new key is inserted. One is that all keys in the memory must remain in ascending order and the other is that there must be no empty locations between the original hash location of any key and its actual storage position. These imply that all keys sharing the same initial hash location must form a single unbroken group.

Within these constraints one would like to insert a new key so as to minimize later retrieval times and the time to do the insertion itself. Intuitively, keys which share the same initial hash location should be centered around that initial address. There are two ways of inserting keys which cause little disturbance to the memory. One is to find the position where the key should be placed according to its ordering and then to create a gap for it by moving *up* all entries from this position up to the next empty location. The second way is symmetric to this and creates a gap by moving entries *down* one location. The insertion algorithm given by Amble and Knuth [1] chooses which of these two moves to make using a strategy which is guaranteed to minimize the number of locations in $T$ which is examined during later successful or unsuccessful searches, although it is not guaranteed to minimize the insertion time itself.

One consequence of this insertion strategy is that sometimes it is necessary to move entries below 0 and above $M$ in the array $T$. One solution to this would be to make the array circular and move entries from 0 to $M - 1$ and vice versa. However, following Amble and Knuth [1], I will instead extend the array $T$ and other arrays to be defined later at their top and bottom. This gives "breathing room" for the array to expand. An extra 20 entries at the top and bottom were found to be quite sufficient for all the simulation runs reported in Section IV. Accordingly, I will define $lo = -20$ and $hi = M + 19$ and define the array $T$ over the range $lo$–$hi$.

## III. Compact Hashing Using Bidirectional Linear Probing

I will now show that the memory required to store the keys in BLP can be significantly reduced. First consider the case when the number of possible keys in $\mathbf{K}$ is less than $M$, then every possible key can be assigned its own location in $T$ without possibility of collision. In this case $T$ degenerates to an ordinary indexed array and the keys need never be stored. At worst, a single bit might be needed to say whether a particular key has been stored or not. This raises the question of whether it is necessary to hold the entire key in memory if the key space $\mathbf{K}$ is slightly larger than $M$. For example, if $\mathbf{K}$ were, say, four times larger than $M$ then it might be possible to hold only two bits of the key rather than the entire key. The reasoning here is that the main function of the stored keys is to ensure that entries which collide at the same location can be correctly separated. Provided $h$ is suitably chosen, at most four keys can be mapped to a single location. The two bits might then be sufficient to store four different values for these four keys. It is in fact possible to realize this reduction in stored key size although a fixed amount of extra information is needed at each location in order to correctly handle collisions.

So that I can talk about the part of the key which is in excess of the address space, I will now introduce a *remainder function* $r$. $r$ maps from the transformed keys $\mathbf{H}$ to a set of remainders $\mathbf{R} = \{0, 1, 2, \cdots, Rm - 1\}$. It is these remainders that will be stored in lieu of the transformed keys. The essential property of $r$ is that $r(H)$ and $h(H)$ together are sufficient to uniquely determine $H$.

Formally,

$$\mathbf{R} = \{0, 1, 2, \cdots, Rm - 1\}$$

$$\mathrm{r}: \mathbf{H} \rightarrow \mathbf{R}$$

and

$$h(H_1) = h(H_2) \quad \textbf{and} \quad r(H_1) = r(H_2) \quad \textbf{iff} \; H_1 = H_2 .$$

For a given function $h$ there are usually many possible functions $r$. One particularly simple pair of functions, referred to by Maurer and Lewis [10] as the *division method,* is $h(H) = \lfloor H/Rm \rfloor$ and $r(H) = H \bmod Rm$.

When $r$ is defined as above and $Rm$ is between $2^d$ and $2^{d+1}$ the number of bits needed to specify a remainder is the number of bits in the key less $d$.

Consider a new array $R[lo \cdots hi]$ into which the remainders will be stored. In what follows $R$ will be kept in place of $T$ but it will be useful to talk about $T$ as if it were still there. $R$ and the additional arrays to be introduced shortly specify just the information in $T$, albeit more compactly. Each value $R[i]$ will hold the value $r(T[i])$ with the exception that when $T[i]$ is $-1$ (marking an empty location) then $R[i]$ is also set to $-1$. If there have been no collisions then each $R[i]$ paired with the value $i$ unambiguously gives the transformed key that would have been stored in $T[i]$. However, if there have been collisions it is not possible to tell if a value of $R[i]$ is at its home location or if it has been moved from, say, $i - 1$ and corresponds to a key $H$ where $r(H) = R[i]$ and $h(H) = i - 1$. If there were some way to locate for each $R[i]$ where it was originally hashed then the original keys could all be unambiguously determined. This can be done by maintaining two extra arrays of bits, the virgin array $V$, and the change array $C$.

The virgin array $V[lo \cdots hi]$ marks those locations which

have never been hashed to. That is, $V[i]$ has a value of 1 stored if any of the stored keys in the hash table has $i$ as its hash address, and 0 otherwise. $V$ is maintained by initializing it to 0 and thereafter setting $V[h(H)] \leftarrow 1$ whenever a key $H$ is inserted in the memory. The virginity of a location is unaffected by the move operations during insertion. The $V$ array is similar to the array of pass bits recommended in [1].

To understand the change array $C[lo \cdots hi]$ it is necessary to look more closely at the distribution of values of $R[i]$. These remainders can be grouped according to whether or not they share the same original hash address. Also recall that the hash table, as in BLP, is ordered, so all the remainders in a particular group will occur at consecutive locations. The change bits $C[i]$ are used to delimit the boundaries of these groups. This is done by marking the first remainder (the one stored at the lowest address) of each group with a 1. All other members of a group have $C[i] = 0$. To simplify the search and insertion algorithms it is also convenient to set $C[i]$ to 1 for all locations which are empty ($R[i] = -1$). Thus, we have the formal definitions of the values of $V$ and $C$ in terms of the now notional array $T$ (the array $A$ is described later)

$$R[i] \equiv \begin{cases} r(T[i]) & T[i] \neq -1 \\ -1 & T[i] = -1 \end{cases}$$

$$V[i] \equiv \begin{cases} 1 & \exists j\ h(T[j]) = i \\ 0 & \text{otherwise} \end{cases}$$

$$C[i] \equiv \begin{cases} 1 & T[i] \neq T[i-1] \quad \textbf{or} \quad T[i] = -1 \\ 0 & \text{otherwise} \end{cases}$$

$$A[i] \equiv \begin{cases} a(i) & -Na \leq a(i) \leq Na \\ \infty & \text{otherwise} \end{cases}$$

where

$$Na \geq 0$$

$$a(i) \equiv \sum_{j=lo}^{i} |C[j] = 1 \quad \text{and} \quad R[j] \neq -1| - \sum_{j=lo}^{i} V[j].$$

### A. Searching

For every group of remainders there will somewhere be a $V$ bit equal to 1 and a $C$ bit at a nonempty location equal to 1. That is, for every $V$ bit which is 1 there is a corresponding $C$ bit which is also 1. This correspondence is indicated in Fig. 1 by the dotted lines. When searching for a key $H$ in the table the location $h(H)$ is examined. If the $V$ bit is 0 then the search can stop immediately. Otherwise, a search is made for the corresponding $C$ bit which is 1. To do this a search is made down (or up) the hash table until an empty location is found. The number of $V$ bits which are 1 from $h(H)$ to this empty location are counted. The correct $C$ bit is then found by counting back up (or down) the array from the empty location for the same number of $C$ bits which are 1. Details of this algorithm follow.

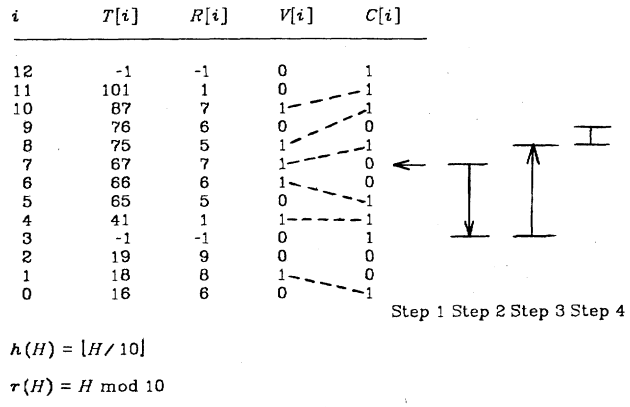


Fig. 1. Example of compact hash memory and search for key.

Step 1: {initialize variables}
$H \leftarrow t(K)$; $j \leftarrow h(H)$; $rem \leftarrow r(H)$; $i \leftarrow j$;
$count \leftarrow 0$;
{check virgin bit}
**if** $V[j] = 0$ **then** {search unsuccessful} **exit**;

Step 2: {find first empty location down the table}
**while** $R[i] \neq -1$ **do begin** $count \leftarrow count - V[i]$;
$i \leftarrow i - 1$ **end**;

Step 3: {search back to find uppermost member of relevant group}
**while** $count < 0$ **do begin** $i \leftarrow i + 1$;
$count \leftarrow count + C[i]$; **end**;
{$i$ now points at the first (lowest) member of the group associated with the original location $j$}

Step 4: {search group associated with $j$}
**while** $R[i+1] \leq rem$ **and** $C[i+1] = 0$
**do** $i \leftarrow i + 1$;
{check last location to see if key found}
**if** $R[i] = rem$ **then** {search successful}
**else** {search unsuccessful};

An example search is illustrated in Fig. 1 for the key 75. For this example, $h$ is computed by dividing by 10 and rounding down, $r$ is computed by taking the remainder modulo 10.

*Step 1:* The initial hash location for 75 is 7 and its remainder is 5. The $V$ bit at location 7 is 1 so the search continues.

*Step 2:* The first empty location found by searching down the table is at location 3. There are three $V$ bits with a value of 1 between 7 and 3 at locations 4, 6, and 7.

*Step 3:* Counting back from location 3 three $C$ bits are 1 at locations 4, 5, and 8. So the $C$ bit at location 8 corresponds to the $V$ bit at the original hash location 7.

*Step 4:* The group of remainders which share the same initial location 7 can then be found in locations 8 and 9. Thus, the remainder 5 at location 8 can be unambiguously associated with the original key 75 and so it can be concluded that the information associated with the key 75 is present at location 8 in the memory.

It still remains to specify the update algorithm and to address some issues of efficiency. To this end, a third array will be added.

### B. Speeding Up Search

It was found during the simulations reported in Section IV that the most time consuming element of this search is Step 2 when the table is scanned for an empty location. The essential role played by the empty locations here is to provide a synchronization between the 1 bits in the $V$ and $C$ arrays. This lengthy search could be eliminated by maintaining two additional arrays, $\#C[lo \cdots hi]$ and $\#V[lo \cdots hi]$, which count from the start of memory the number of $C$ and $V$ bits which are 1. That is,

$$\#C[i] \equiv \sum_{j=lo}^{i} |C[j] = 1 \quad \textbf{and} \quad R[j] \neq -1|$$

and

$$\#V[i] \equiv \sum_{j=lo}^{i} V[j].$$

In order to find the $C$ bit corresponding to some $V[i] = 1$ then all that is necessary is to compute the difference $count \leftarrow \#C[i] - \#V[i]$. If *count* is zero then the remainder stored at $i$ was originally hashed there and has not been moved. If *count* is positive then it is necessary to scan down the memory until '*count*' $C$ bits equal to 1 have been found. If *count* is negative then it is necessary to scan up the memory until '$-$*count*' $C$ bits which are 1 have been found. Fig. 2 shows some examples of the various situations which can arise.

In fact, it is not necessary to store $\#C$ and $\#V$ explicitly; it is sufficient merely to store the differences $\#C[i] - \#V[i]$. To do this the *At home* array $A[lo \cdots hi]$ will be used.

At this point it might seem that all earlier gains have been lost because in the most extreme case $\#C[i] - \#V[i] = M$. To store a value of $A$ will require as many bits as a memory index — precisely the gain made by storing remainders rather than keys! However, all is not lost. The values of $A$ tend to cluster closely about 0. Simulation shows that a hash memory which is 95 percent full has 99 percent of the $A$ values in the range $-15$ to $+15$. Therefore, the following strategy can be adopted. Assign a fixed number of bits for storing each value of $A$, say 5 bits. Use these bits to represent the 31 values $-15$ to $+15$ and a 32nd value for $\infty$. Then anywhere that $\#C[i] - \#V[i] < -15$ *or* $> +15$ assign $\infty$ to $A[i]$; otherwise assign the true difference.

When searching for a key, a scan can now be done down (or up) the memory until a location $i$ where $A[i] \neq \infty$ is found. (At worst this will occur at the first unoccupied location where $A[i]$ will be zero.) From there a count can be made up (or down) the memory for the appropriate number of $C$ bits which is 1.

In the detailed algorithm given below some differences from the simpler search can be noted. In Step 3, *count* can be both positive and negative. Therefore, code is included to scan both up and down the memory as appropriate. At the end of Step 3, $i$ can be pointing at any member of the group associated with the original hash location. (Above, $i$ was always left pointing at the lowest member of the group.) Therefore, code is included for scanning both up and down the members of the group. In order to prevent redundant checking of locations by this code, a flag *direction* is used. It can take on three values depending on the direction of the memory scan: *up*, *down*, and *here* (no further searching need be done).

```
{Search using at-home count}
Step 1: {initialize variables}
        H ← t(K); j ← h(H); rem ← r(H); i ← j;
        count ← 0;
        {check virgin bit}
        if V[j] = 0 then {search unsuccessful} exit;
Step 2: {find first well defined A value down the memory}
        while A[i] = ∞ do begin count ← count − V[i];
          i ← i − 1 end;
        count ← count + A[i];
Step 3: {Search either up or down until a member of sought
          group is found}
        {Also ensure direction is set for Step 4.}
        if count < 0 then
                direction ← up;
                repeat i ← i + 1; count ← count + C[i]
                  until count = 0;
                if R[i] ≥ rem then direction ← here;
        else if count > 0 then
                direction ← down;
                repeat count ← count − C[i]; i ← i − 1
                  until count = 0;
                if R[i] ≤ rem then direction ← here;
        else {count = 0}
                if R[i] > rem then direction ← down
                else if R[i] < rem then direction ← up
                else direction ← here;
Step 4: {search group associated with j}
        case direction of
        here:  ;{do nothing}
        down: repeat if C[i] = 1 then direction ← here
                      else
                      begin
                            i ← i − 1;
                            if R[i] ≤ rem then
                              direction ← here;
                      end
              until direction = here;
        up:   repeat if C[i + 1] = 1 then
                            direction ← here
                      else
                      begin
                            i ← i + 1;
                            if R[i] ≥ rem then
                              direction ← here;
                      end
              until direction = here;
        end;
Step 5: {check last location to see if key found}
        if R[i] = rem then {search successful}
          else {search unsuccessful};
```

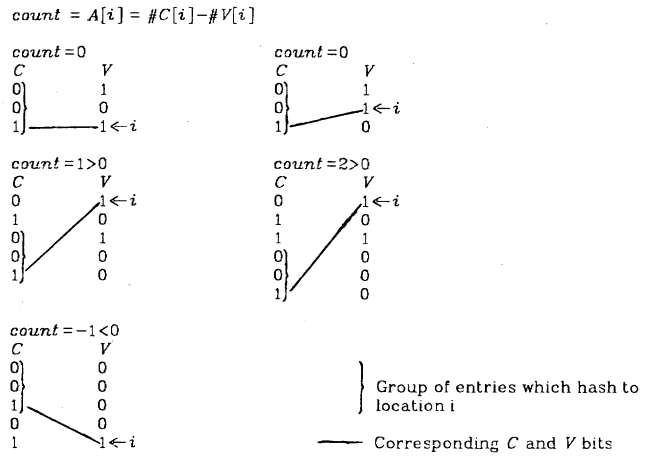


Fig. 2. Examples showing different values of $\#C[i] - \#V[i]$.

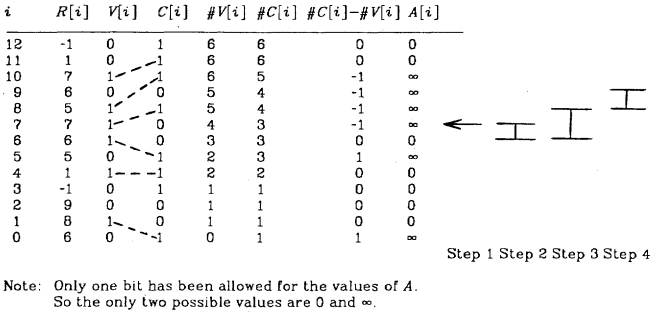


Fig. 3. Example of calculation and use of array $A$.

Fig. 3 gives an example of this searching algorithm. The same memory and key (75) as in Fig. 1 are used. For the purposes of the example each $A$ value is allocated one bit. This allows only two values, 0 and $\infty$. The search proceeds as follows.

*Step 1:* The initial hash location for 75 is 7 and its remainder is 5. The $V$ bit at location 7 is 1 so the search continues.

*Step 2:* The first $A$ value not equal to $\infty$ found by searching down the table is at location 6. There is one $V$ bit with a value of 1 between 7 and 6, at location 7. *count* is then set to $A[6] + 1 = 1$. So on the next step one $C$ bit will be sought.

*Step 3:* Counting back up from 6, the first $C$ bit equal to 1 is at location 8. So the $C$ bit at location 8 corresponds to the $V$ bit at the original hash location 7.

*Step 4:* The group of remainders which share the same initial location 7 can then be found in locations 8 and 9. The remainder 5 at location 8 can thus be unambiguously associated with the original key 75 and it can be concluded that the information associated with the key 75 is present at location 8 in the memory.

### C. Insertion

Insertion of a new key into the memory requires three distinct steps: first, locating whereabouts in the memory the key is to be placed; second, deciding how the memory is to be rearranged to make room for the new key; and third, moving the remainders while correctly preserving the $A$ and $C$ values. (The $V$ bits remain fixed during the move.) The initial search can be done as explained above with the small addition that the correct insertion point must still be located when the key is not present. The second and third steps follow the algorithm in Amble and Knuth [1] with the addition that the values of the $A$ array must be recalculated over the shifted memory locations and the $C$ but not the $V$ bits must be moved with the keys. Details of this can be found in an earlier draft of this paper [4].

## IV. Performance

Now I consider how long these algorithms will take to run. The measure of run time that I will use is the number of *probes* that each algorithm makes, that is, the number of times locations in the hash table are examined or updated. CPU time measures were taken as well and correlate well with the empirical counts of probes given below.

### A. Searching

Tables I and II list the results of simulations for successful and unsuccessful searches, respectively. Results are tabulated for ordinary BLP and for compact hashing with different memory loadings and different sizes for the $A$ field. If the number of keys stored in the memory is $N$, then the memory loading is measured by $\alpha = N/M$, the fraction of locations in the memory which are full. Values of Na were chosen to correspond to $A$ field lengths of 1, 2, 3, 4, and 5 bits, that is for Na equal to 0, 1, 3, 7, and 15, respectively, and also for the case where no $A$ field was used. Increasing the size of the $A$ field beyond 5 bits had no effect at the memory loadings investigated. So Na equal to 15 is effectively the same as an unbounded size for the $A$ values.

The insertion procedure is guaranteed to be optimum only for BLP, not for compact hashing. If none of the values in $A$ is $\infty$ then the sequence of locations examined by compact hashing is the same as for BLP and so the strategy will still be optimum. (This is easily seen by noting that in compact hashing $A[h(t(K))]$ determines the direction of the search depending on whether it is positive or negative. During the subsequent search no locations past the sought key will be probed. This is exactly the same probing behavior as in BLP.) However, if no $A$ array is being used or if some values of $A$ are $\infty$, then extra locations need to be probed to find an empty location or one which is not equal to $\infty$.

As expected, the figures in Table I show that for Na at 15 and using optimum insertion the probes for a successful search are almost the same as for BLP. (The small differences are accounted for by statistical fluctuations in the simulation results.)

As Na is decreased the number of probes needed for searching increases. This reflects the greater distances that must be traversed to find a value of $A$ not equal to $\infty$. It is notable however that even a single bit allocated to the $A$ fields dramatically improves the performance. Even at a memory density of 0.95 some 25 percent of nonempty locations have $A$ values at 0.

The pattern for unsuccessful searches is broadly the same as sketched above for successful searches except that in gen-

TABLE I
AVERAGE NUMBER OF PROBES DURING A SUCCESSFUL SEARCH

| α | .25 | .5 | .75 | .8 | .85 | .9 | .95 |
|---|---|---|---|---|---|---|---|
| *BLP*[1] | 1.1 | 1.3 | 1.7 | 2.0 | 2.3 | 2.9 | 4.2 |
| Na | | | | | | | |
| 15 | 1.1 | 1.3 | 1.7 | 1.9 | 2.2 | 2.8 | 4.6 |
| 7 | 1.1 | 1.3 | 1.7 | 1.9 | 2.2 | 2.8 | 9.7 |
| 3 | 1.1 | 1.3 | 1.7 | 1.9 | 2.4 | 4.2 | 25 |
| 1 | 1.1 | 1.3 | 2.0 | 2.5 | 4.1 | 8.8 | 45 |
| 0 | 1.1 | 1.5 | 3.3 | 4.9 | 7.9 | 15 | 61 |
| - | 4.2 | 7.1 | 20 | 30 | 49 | 110 | 370 |
| * | 3.77 | 6.00 | 18.0 | 27.0 | 46.4 | 102 | 402 |

[1] Taken from Amble and Knuth [1].
- No *A* field used.
* Analytic approximation to line above.

TABLE II
AVERAGE NUMBER OF PROBES DURING AN UNSUCCESSFUL SEARCH

| α | .25 | .5 | .75 | .8 | .85 | .9 | .95 |
|---|---|---|---|---|---|---|---|
| *BLP*[1] | 1.3 | 1.5 | 2.1 | 2.3 | 2.6 | 3.1 | 4.4 |
| Na | | | | | | | |
| 15 | 1.2 | 1.4 | 1.8 | 1.9 | 2.1 | 2.4 | 3.5 |
| 7 | 1.2 | 1.4 | 1.8 | 1.9 | 2.1 | 2.4 | 9.7 |
| 3 | 1.2 | 1.4 | 1.8 | 1.9 | 2.2 | 3.3 | 15 |
| 1 | 1.2 | 1.4 | 1.9 | 2.2 | 3.2 | 6.0 | 28 |
| 0 | 1.2 | 1.5 | 2.6 | 3.4 | 5.3 | 9.9 | 36 |
| - | 1.7 | 3.4 | 11 | 16 | 28 | 64 | 220 |
| * | 1.72 | 3.16 | 10.2 | 15.6 | 27.3 | 61.2 | 247 |

[1] Taken from Amble and Knuth [1].
- No *A* field used.
* Analytic approximation to line above.

eral unsuccessful searches are quicker than successful ones. This is a result of the *V* bits which allow many unsuccessful searches to be stopped after a single probe. For example, even at the maximum possible memory density of 1 some 36 percent of *V* bits are zero. This results in compact hashing being faster than the reported values for ordinary BLP. However, unsuccessful BLP searches could be improved to a similar degree by incorporating *V* bits.

### B. Insertion

The probes to insert a new key can be broken down into three components: those needed to locate the position where the key is to be inserted, those to decide the direction of movement, and those to effect the movement of the memory. The first of these will be slightly larger than a successful search and so the results of Table I have not been repeated. The second two are independent of Na as they are dependent only on the lengths of blocks of adjacent nonempty locations. The values for these Na independent components are listed in Table III. In most cases, this Na independent component is much larger than the search component. The exception occurs where no *A* values are being used, when the two components are comparable.

Cleary [5] examines a random insertion strategy for ordinary BLP where blocks of entries in the hash table are moved in a randomly chosen direction to accommodate a new entry rather than in the optimum way described by Amble and Knuth [1]. It is shown that this strategy can improve insertion times by a factor of 4 at the expense of small degradations (at most 15 percent) in retrieval times. These results are shown by simulation to extend to compact hashing. Indeed, for small values of Na the optimum and random strategies show no significant differences in retrieval times.

TABLE III
AVERAGE NUMBER OF PROBES TO MOVE BLOCK OF MEMORY

| α | .25 | .5 | .75 | .8 | .85 | .9 | .95 |
|---|---|---|---|---|---|---|---|
| | 4.3 | 6.8 | 32 | 49 | 86 | 200 | 700 |
| * | 4.56 | 9.00 | 33.0 | 51.0 | 89.9 | 201 | 801 |

* Analytic approximation to line above

### C. Analytic Approximation

While analytic results are not available for the number of probes needed for retrieval or insertion, an approximation can be developed for some of the cases. It is shown by Amble and Knuth [1] and Knuth [8, problem 6.4-47] that the average length of a block of consecutive nonempty locations when using the optimum insertion strategy is approximately $(1 - \alpha)^{-2} - 1$. Let this block length be $L$.

Consider the case of a successful search when no *A* field is used. A successful scan of a block from an arbitrary position to the end takes on average $L/2 + 1/2$ probes. During the initial scan down the memory in the simulations the initial check of the *V* bit and the final empty location examined were each counted as a single probe. This gives a total of $L/2 + 5/2$ probes for the initial scan down. (This is not exact because there will be a correlation between the position of a key's home location within a block and the number of keys hashing to that home location.) The scan back up a block will take $L/2 + 1/2$ probes (exact for a successful search). This gives $(1 - \alpha)^{-2} + 2$ for the expected number of probes during a successful search. These values are listed in Table I and are consistently low by about 10 percent.

For an unsuccessful search with no *A* field, the initial scan down the memory will take $L/2 + 5/2$ probes as above (again, this will not be exact because the probability of a *V* bit being one will be correlated with the size of a block and its position within the block). An unsuccessful scan of a block takes $L/2 + 1/2$ probes. (This assumes the keys in the block are distributed uniformly. This gives the following probabilities that the search will stop at a particular location in the block: the first location, $1/2L$; locations 2–$L$, $1/L$; the empty $(L + 1)$st location, $1/2L$. This will not be true for compact hashing because the probability of stopping at a key which shares its home location with a large number of other keys will be smaller than for one which shares it with few others.) Summing these two terms gives $L + 7/2$ probes. Given that the keys are distributed randomly, there is a probability of $e^{-\alpha}$ that a given *V* bit will be zero. So the expected number of probes overall for an unsuccessful search is $e^{-\alpha} + (1 - e^{-\alpha}) \cdot ((1 - \alpha)^{-2} + 5/2)$. These values are listed in Table II and are consistently low by about 5 percent.

Considering only the insertion component which is independent of Na then it is possible to derive an expression for the number of probes. There is an initial scan to move the memory down and insert the new key which will scan about

half the block ($L/2 + 5/2$ probes) and a subsequent scan back up the entire block ($L + 1$ probes). Empirically, the probability that the entire block will subsequently be moved back up is a half which gives an expected $1/2(L + 1)$ probes. Summing these three contributions gives $2(1 - \alpha)^{-2} + 2$ as the expected number of probes for an insertion (excluding the search time). Values for this expression are tabulated in Table III; they are in good agreement with the empirical values.

## Acknowledgment

I would like to thank I. Witten for careful checking of a draft version, and J. Andreae for discussions which showed that something like compact hashing might be possible.

## References

[1] O. Amble and D. E. Knuth, "Ordered hash tables," *Comput. J.*, vol. 17, pp. 135–142, 1974.
[2] J. H. Andreae, *Thinking with the Teachable Machine*. London: Academic, 1977.
[3] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *J. Comput. Syst. Sci.*, vol. 18, pp. 143–154, 1979.
[4] J. G. Cleary, "Compact hash tables," Dep. Comput. Sci., Univ. Calgary, Calgary, Alta., Canada, Res. Rep. 82/100/19, July 1982.
[5] ——, Random insertion for bidirectional linear probing can be better than optimum," Dep. Comput. Sci., Univ. Calgary, Calgary, Alta., Canada, Res. Rep. 82/105/24, Sept. 1982.
[6] J. A. Feldman and J. R. Low, "Comment on Brent's scatter storage algorithm," *Commun. Ass. Comput. Mach.*, vol. 16, p. 703, 1973.
[7] G. D. Knott, "Hashing functions," *Comput. J.*, vol. 18, pp. 265–278, 1975.
[8] D. E. Knuth, *The Art of Computer Programming: Sorting and Searching, Vol. III*. Reading, MA: Addison-Wesley, 1973.
[9] V. Y. Lum, "General performance analysis of key-to-address transformation methods using an abstract file concept," *Commun. Ass. Comput. Mach.*, vol. 16, pp. 603–612, 1973.
[10] V. Y. Lum, P. S. T. Yuen, and M. Dodd, "Key-to-address transformation techniques," *Commun. Ass. Comput. Mach.*, vol. 14, pp. 228–239, 1971.
[11] W. D. Maurer and T. G. Lewis, "Hash table methods," *Comput. Surveys*, vol. 7, pp. 5–19, 1975.

**John G. Cleary** received the B.Sc.(hons) degree in 1971, the M.Sc. degree in 1974, both in mathematics, and the Ph.D. degree in 1980 in electrical engineering, all from the University of Canterbury, New Zealand.

From 1975 to 1981 he worked for Progeni Systems Ltd., Wellington, New Zealand. In 1981 he was a Lecturer in Computer Science at the Victoria University, Wellington. In 1982 he moved to the University of Calgary, Calgary, Alberta, Canada, where he is currently an Assistant Professor. His current research interests include adaptive systems and their applications in man–machine interfaces, logic programming and formal specification, and applications of parallel processor systems.